Assignment 3 - Technical Report

Edge Computing & Intelligent Systems | Model quantization (PTQ, QAT) and pruning (unstructured, structured) of ResNet-18 on CIFAR-10

Name: Rana Raza Ahmed Roll No: 23i-0060 Section: AI-B Date: 13-Nov-2026

1. Overview and setup

This report documents a complete PyTorch compression pipeline for one image classifier. A single FP32 ResNet-18 is trained on CIFAR-10 (Q1.1 / Q2.1) and then compressed four ways: post-training static quantization (PTQ), quantization-aware training (QAT), iterative unstructured magnitude pruning and structured channel pruning with physical architecture reconstruction. All five models are served by one inference pipeline, evaluated by automated benchmark scripts and exported to ONNX for CPU deployment.

Item	Value
Dataset	CIFAR-10: 50,000 train / 10,000 test images, 10 classes, 32x32 RGB
Model	ResNet-18 for 32x32 input: 3x3 stride-2 stem, no max-pool, 4 stages x 2 BasicBlocks (64-128-256-512), 11,173,962 parameters, 138.9 M MACs
Hardware	CPU only (2 vCPU, no GPU); all latency numbers are CPU numbers
Software	Python 3.13, PyTorch 2.15, torchvision 0.29, ONNX 1.23, ONNX Runtime 1.29
Quantization backend	fbgemm (x86); the code falls back to qnnpack on ARM
Seed	42 for all scripts
Test set use	Used only for reporting; no model selection or tuning was done on it

Design note. Because training had to run on a 2-core CPU, the stem uses stride 2 (feature maps 16-8-4-2 instead of 32-16-8-4), which cuts compute about 4x versus the usual CIFAR ResNet-18. The baseline was trained for only 8 epochs, so it is deliberately an under-trained model (86.33%); this matters for interpreting results in Section 3.4.

2. Baseline FP32 model (Deliverable 1)

Script: `src/train_baseline.py`. SGD (Nesterov, momentum 0.9), batch 128, one-cycle LR schedule with peak 0.1, weight decay 5e-4, random-crop (pad 4) + horizontal-flip augmentation, 8 epochs (about 47 min on CPU). Final test accuracy: **86.33%**. Checkpoint: `checkpoints/fp32_resnet18.pth` (44.8 MB).

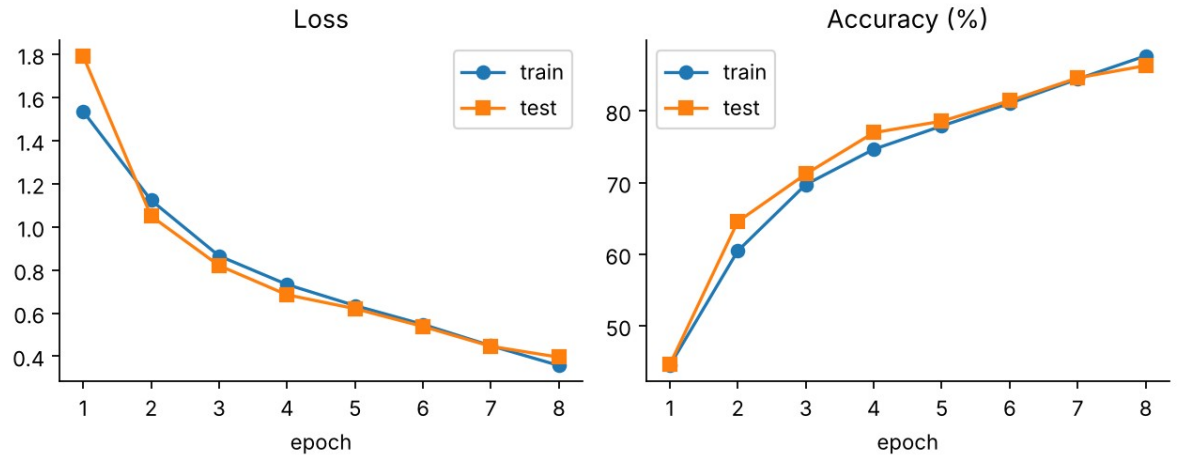


Figure 1 - Baseline training curves

```
$ python src/train_baseline.py --epochs 8

=== Baseline training | {'epochs': 8, 'batch_size': 128, 'lr': 0.1, 'wd': 0.0005, 'seed': 42, 'data_dir':
'/home/claude/sub/data', 'out': '/home/claude/sub/checkpoints/fp32_resnet18.pth', 'threads': 0}
parameters: 11,173,962
epoch  1/8 train_loss 1.5360 train_acc 44.48 | test_loss 1.7932 test_acc 44.61 | 4.7 min
epoch  2/8 train_loss 1.1236 train_acc 60.47 | test_loss 1.0503 test_acc 64.56 | 18.0 min
epoch  3/8 train_loss 0.8654 train_acc 69.73 | test_loss 0.8204 test_acc 71.20 | 22.9 min
epoch  4/8 train_loss 0.7325 train_acc 74.65 | test_loss 0.6849 test_acc 76.97 | 27.7 min
epoch  5/8 train_loss 0.6346 train_acc 77.90 | test_loss 0.6205 test_acc 78.57 | 32.5 min
epoch  6/8 train_loss 0.5475 train_acc 81.07 | test_loss 0.5378 test_acc 81.43 | 37.4 min
epoch  7/8 train_loss 0.4483 train_acc 84.46 | test_loss 0.4458 test_acc 84.62 | 42.3 min
epoch  8/8 train_loss 0.3569 train_acc 87.70 | test_loss 0.3954 test_acc 86.33 | 47.0 min
Saved FP32 checkpoint -> /home/claude/sub/checkpoints/fp32_resnet18.pth  (final test acc 86.33%)
```

Screenshot 1 - Baseline training console output

3. Question 1 - Quantization

3.1 Model preparation

The ResNet was written to be quantization-friendly: `QuantStub/DeQuantStub` at the input/output, and the residual add+ReLU is done with `FloatFunctional.add_relu` so that it can be quantized in eager mode. Conv+BN(+ReLU) groups are fused (`fuse_model` in `common.py`): stem, both convs of each block and the 1x1 shortcut convs. Fusion was verified to leave FP32 outputs unchanged.

3.2 Post-training static quantization (Deliverable 2)

Script: `src/ptq.py`. Steps: load the FP32 checkpoint, fuse, select the backend (`torch.backends.quantized.engine = "fbgemm"`), attach the default fbgemm qconfig (per-channel INT8 weights, histogram observers for activations), insert observers with `prepare`, run the calibration pipeline over 1,000 randomly chosen training images (no augmentation, forward-only), then `convert` to a real INT8 model. The model is saved as TorchScript (`checkpoints/ptq_int8_resnet18.pt`, 11.4 MB) and re-loaded to confirm identical accuracy.

Result: **86.30%** vs 86.33% FP32 (-0.03 points) at 3.9x smaller size. A calibration-size sweep (10 to 5,000 images) gave 86.23-86.41% in every case, so PTQ accuracy is insensitive to calibration size for this model; 1,000 images is therefore a safe default.

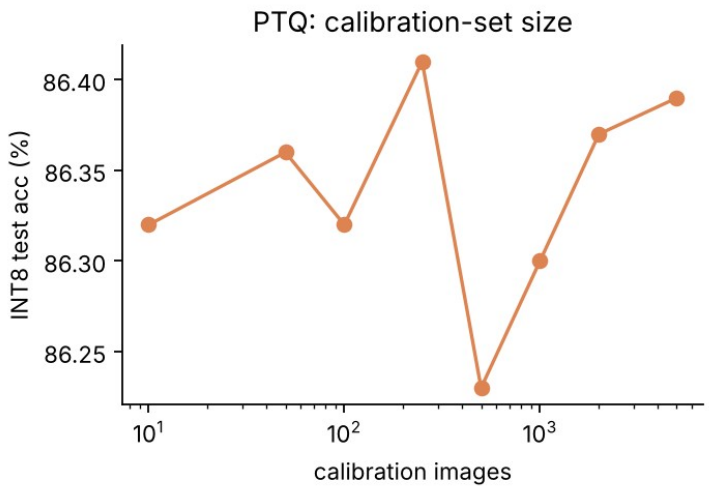


Figure 2 - PTQ accuracy vs calibration-set size

```
$ python src/ptq.py --calib-size 1000 --ablation

=== PTQ | backend=fbgemm | {'calib_size': 1000, 'fp32': '/home/claude/sub/checkpoints/fp32_resnet18.pth', 'out':
'/home/claude/sub/checkpoints/ptq_int8_resnet18.pt', 'data_dir': '/home/claude/sub/data', 'ablation': True, 'seed':
42}
FP32 test accuracy: 86.33%   (44.8 MB)
Calibrating with 1000 training images ...
PTQ INT8 test accuracy: 86.30%  (drop +0.03 pts)
Saved deployable INT8 model -> /home/claude/sub/checkpoints/ptq_int8_resnet18.pt (11.4 MB); re-loaded accuracy 86.30%
  calibration size    10 -> INT8 acc 86.32%
  calibration size    50 -> INT8 acc 86.36%
  calibration size   100 -> INT8 acc 86.32%
  calibration size   250 -> INT8 acc 86.41%
  calibration size   500 -> INT8 acc 86.23%
  calibration size  1000 -> INT8 acc 86.30%
  calibration size  2000 -> INT8 acc 86.37%
  calibration size  5000 -> INT8 acc 86.39%
```

Screenshot 2 - PTQ console output

3.3 Quantization-aware training (Deliverable 3)

Script: `src/qat.py`. The FP32 checkpoint initialises the model, which is fused with the QAT-aware fusion and prepared with `prepare_qat` (51 fake-quant modules inserted: weights per-channel, activations per-tensor). It is fine-tuned for 2 epochs (780 steps) with SGD, LR 0.01 cosine-annealed, weight decay 1e-4. Following the standard recipe, BatchNorm statistics are frozen after 60% of the steps and observers after 80%. The model is then converted to INT8 and exported as TorchScript (`checkpoints/qat_int8_resnet18.pt`, 11.4 MB).

Result: fake-quant model 87.18%, converted INT8 model **87.30%**.

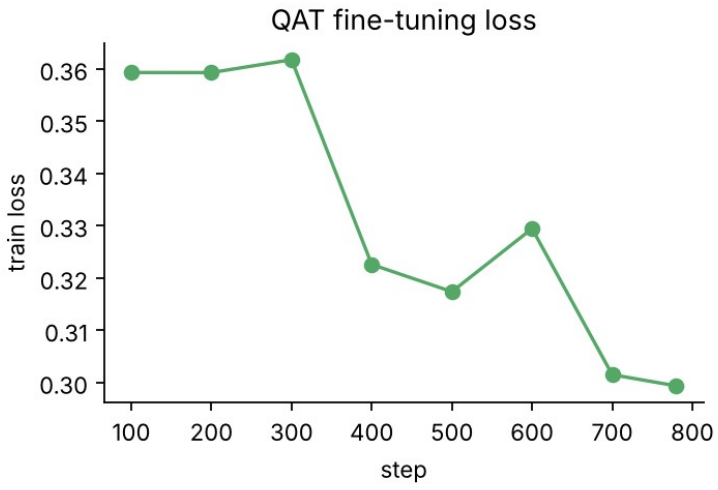


Figure 3 - QAT fine-tuning loss

```
$ python src/qat.py --epochs 2

=== QAT | backend=fbgemm | {'epochs': 2.0, 'batch_size': 128, 'lr': 0.01, 'wd': 0.0001, 'freeze_bn_frac': 0.6,
'freeze_obs_frac': 0.8, 'fp32': '/home/claude/sub/checkpoints/fp32_resnet18.pth', 'out':
'/home/claude/sub/checkpoints/qat_int8_resnet18.pt', 'data_dir': '/home/claude/sub/data', 'seed': 42, 'no_onnx':
False}
FP32 starting point: 86.33%
fake-quant modules inserted: 51
step   100/780 loss 0.3593 lr 0.00960 | 1.8 min
step   200/780 loss 0.3593 lr 0.00848 | 3.4 min
step   300/780 loss 0.3618 lr 0.00681 | 5.0 min
step   400/780 loss 0.3226 lr 0.00485 | 6.6 min
step 469: BN statistics frozen
step   500/780 loss 0.3174 lr 0.00293 | 8.3 min
step   600/780 loss 0.3295 lr 0.00134 | 10.0 min
step 625: observers frozen (quantization ranges fixed)
step   700/780 loss 0.3015 lr 0.00035 | 11.7 min
step   780/780 loss 0.2994 lr 0.00010 | 13.0 min
QAT fake-quant model test accuracy: 87.18%
QAT INT8 test accuracy: 87.30%  (vs FP32 86.33%, drop -0.97)
Saved deployable INT8 model -> /home/claude/sub/checkpoints/qat_int8_resnet18.pt (11.4 MB); reloaded acc 87.30%
```

Screenshot 3 - QAT console output

3.4 Inference pipeline and benchmark (Q1.4, Q1.5, Deliverables 6-7)

`src/inference.py` exposes one `Predictor` class for all models and backends (CLI: `--model fp32|ptq|qat|unstructured|structured|all --backend torch|onnx`, optionally on a custom image). `src/benchmark_q1.py` evaluates FP32, PTQ and QAT on the full 10,000-image test set and writes `results/q1_benchmark.csv` (metrics) and `results/q1_predictions.csv` (per-image prediction, confidence and correctness for each model). Latency is batch-1 on one thread (200 timed runs after warm-up); throughput is batch-256 on both cores.

Model	Test acc (%)	Size (MB)	Latency b=1 (ms)	Throughput (img/s)	Agreement with FP32
FP32	86.33	44.78	6.37	861.5	100%
PTQ-INT8	86.30	11.43	1.16	2695.4	98.99%
QAT-INT8	87.30	11.43	1.29	2735.1	95.64%

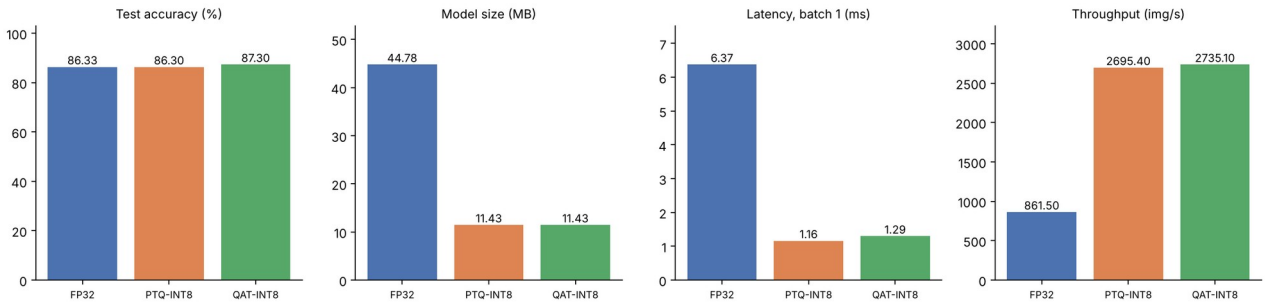


Figure 4 - Q1 comparison of FP32, PTQ-INT8 and QAT-INT8

Analysis. Both INT8 models are 3.9x smaller, about 5x faster per image and 3.1x faster in batch throughput than FP32. PTQ is essentially lossless here. QAT scores about 1 point higher than FP32, but this should not be read as quantization improving accuracy: the 8-epoch baseline is still improving, and QAT gives it two further epochs of low-LR training. A fair statement is that QAT lost nothing relative to the (fine-tuned) float model, while PTQ lost 0.03 points without any retraining. The lower agreement of QAT with FP32 predictions (95.6%) reflects that QAT has moved the weights, not that it is less accurate. PTQ was already near-lossless on this network, so the advantage QAT is designed to give (recovering accuracy on models that are fragile under PTQ) cannot be demonstrated cleanly on it.

```
$ python src/benchmark_q1.py

model  acc%    MB  gzMB  lat_ms  img/s  nz_params  MACs_M  sparse%  lat_x  thr_x
 fp32 86.33 44.78 41.30   6.374  861.5 11173962.0  138.86      0.0   1.00   1.00
  ptq 86.30 11.43 10.43   1.158 2695.4        NaN     NaN      NaN   5.50   3.13
  qat 87.30 11.43 10.44   1.294 2735.1        NaN     NaN      NaN   4.93   3.17
```

Screenshot 4 - Q1 benchmark output

4. Question 2 - Pruning

4.1 Unstructured pruning (Deliverable 4)

Script: `src/prune_unstructured.py`. Every Conv2d and the final Linear layer is a target. Weights are ranked **globally by L1 magnitude** with `torch.nn.utils.prune.global_unstructured`. Pruning is **iterative**: 5 rounds on a cubic sparsity schedule (41.5%, 66.6%, 79.6%, 84.3%, 85.0%); after each round the network is fine-tuned for 0.4 epoch, followed by a final 1-epoch fine-tune with the final mask. Weight masking is done by the pruning re-parametrisation (`weight_orig` and `weight_mask`, re-applied in every forward pass so pruned weights stay zero during fine-tuning). At the end `prune.remove` makes pruning **permanent**: the mask is folded into the weight tensor and the script asserts that no masks remain and that sparsity and accuracy are unchanged.

Round	Target / actual sparsity	Acc right after pruning	Acc after fine-tune
1	41.5%	86.37	86.41
2	66.6%	86.47	86.44
3	79.6%	85.92	86.49
4	84.3%	85.99	86.70
5	85.0%	86.18	86.80
final fine-tune	85.0%	-	86.97

Final model: 85.0% of conv/FC weights are zero (1,684,263 of 11,173,962 parameters remain, 15.1%), **86.97%** accuracy (+0.64 over baseline; again partly the extra fine-tuning epochs). Two files are exported: a dense-format checkpoint that loads into the normal ResNet18 (`pruned_unstructured_resnet18.pth`, 44.8 MB, 10.0 MB after gzip) and a genuinely compact bit-mask-plus-values file (`..._sparse_compact.pth`, 8.2 MB) with a loader whose reloaded accuracy was checked to be identical.

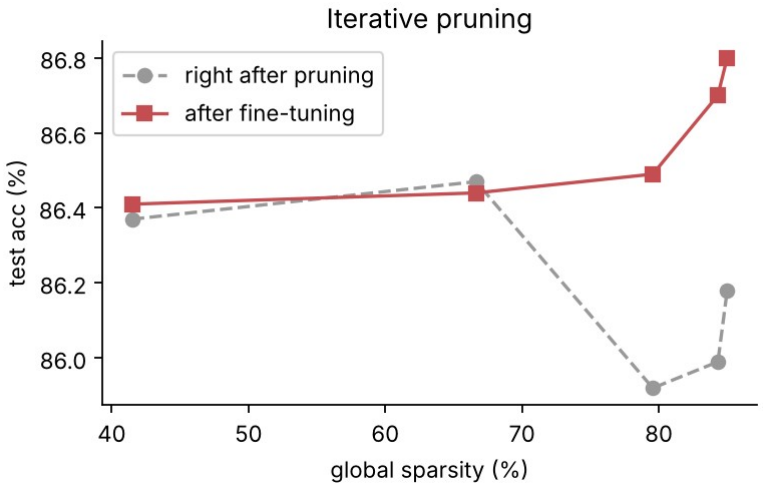


Figure 5 - Accuracy through the iterative pruning schedule

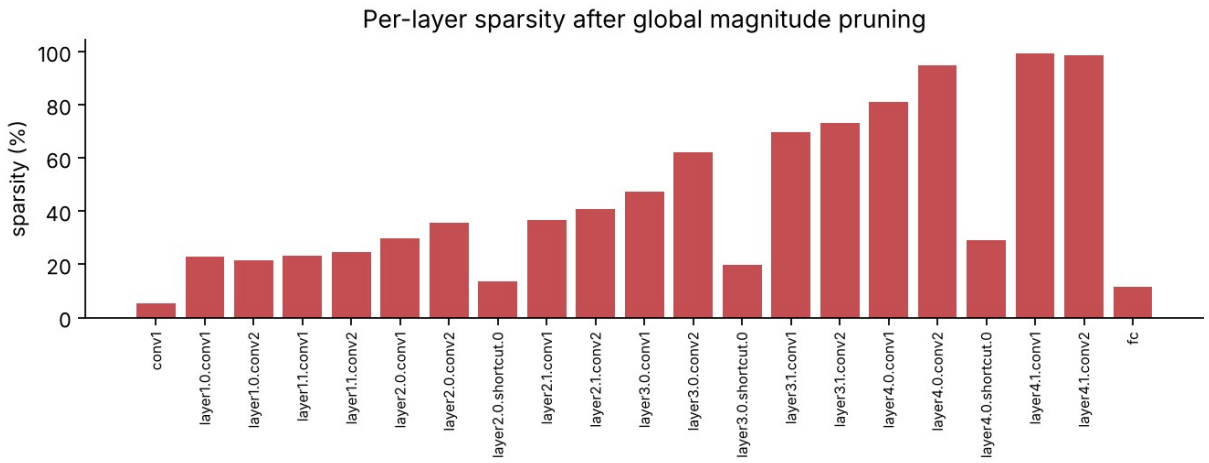


Figure 6 - Per-layer sparsity: global ranking prunes the large late layers hardest and spares the small early ones

```
$ python src/prune_unstructured.py --sparsity 0.85 --iterations 5

=== Unstructured pruning | {'sparsity': 0.85, 'iterations': 5, 'ft_epochs': 0.4, 'final_ft_epochs': 1.0, 'lr': 0.01,
'fp32': '/home/claude/sub/checkpoints/fp32_resnet18.pth', 'out':
'/home/claude/sub/checkpoints/pruned_unstructured_resnet18.pth', 'data_dir': '/home/claude/sub/data', 'seed': 42}
Baseline FP32 accuracy: 86.33%
sparsity schedule: [0.415, 0.666, 0.796, 0.843, 0.85]
[iter 1/5] target 0.415 actual 0.415 | acc right after pruning 86.37%
   ...
  [iter1] done -> test acc 86.41%
[iter 2/5] target 0.666 actual 0.666 | acc right after pruning 86.47%
  [iter2] step  100/156 loss 0.3392 | 1.2 min
  [iter2] step  156/156 loss 0.3420 | 1.9 min
  [iter2] done -> test acc 86.44%
[iter 3/5] target 0.796 actual 0.796 | acc right after pruning 85.92%
  [iter3] step  100/156 loss 0.3322 | 1.3 min
  [iter3] step  156/156 loss 0.3264 | 2.0 min
  [iter3] done -> test acc 86.49%
[iter 4/5] target 0.843 actual 0.843 | acc right after pruning 85.99%
  [iter4] step  100/156 loss 0.3371 | 1.2 min
  [iter4] step  156/156 loss 0.3294 | 1.9 min
  [iter4] done -> test acc 86.70%
[iter 5/5] target 0.850 actual 0.850 | acc right after pruning 86.18%
  [iter5] step  100/156 loss 0.3272 | 1.3 min
  [iter5] step  156/156 loss 0.3223 | 2.0 min
  [iter5] done -> test acc 86.80%
Final fine-tuning with the final mask ...
  [final] step  100/390 loss 0.3305 | 1.3 min
  [final] step  200/390 loss 0.3283 | 2.5 min
  [final] step  300/390 loss 0.3207 | 3.7 min
  [final] step  390/390 loss 0.3067 | 4.8 min
  [final] done -> test acc 86.97%
Made permanent: sparsity 0.8500 -> 0.8500; acc 86.97% -> 86.97%
Parameters: total 11,173,962, non-zero 1,684,263 (15.1% dense)
Saved dense checkpoint  -> /home/claude/sub/checkpoints/pruned_unstructured_resnet18.pth (44.8 MB, gzip 10.0 MB)
Saved compact checkpoint-> /home/claude/sub/checkpoints/pruned_unstructured_resnet18_sparse_compact.pth (8.2 MB);
reload acc 86.97%
per-layer sparsity (%): {'conv1': 5.38, 'layer1.0.conv1': 22.79, 'layer1.0.conv2': 21.41, 'layer1.1.conv1': 23.34,
'layer1.1.conv2': 24.69, 'layer2.0.conv1': 29.67, 'layer2.0.conv2': 35.74, 'layer2.0.shortcut.0': 13.71,
'layer2.1.conv1': 36.7, 'layer2.1.conv2': 40.98, 'layer3.0.conv1': 47.5, 'layer3.0.conv2': 62.22,
'layer3.0.shortcut.0': 19.96, 'layer3.1.conv1': 69.79, 'layer3.1.conv2': 73.08, 'layer4.0.conv1': 81.13,
'layer4.0.conv2': 94.9, 'layer4.0.shortcut.0': 29.22, 'layer4.1.conv1': 99.55, 'layer4.1.conv2': 98.56, 'fc': 11.43}
```

Screenshot 5 - Unstructured pruning console output

4.2 Structured channel pruning (Deliverable 5)

Script: `src/prune_structured.py`. Inside each BasicBlock the hidden layer `conv1 -> bn1 -> ReLU -> conv2` can be thinned freely, whereas the block output channels are tied to the residual stream (identity or 1x1 shortcut) and are therefore kept. Importance of a hidden channel is the L1 norm of its `conv1` filter, normalised by the layer mean so that one global threshold works across layers; every layer keeps at least 25% of its channels. 50% of hidden channels are removed globally.

Keeping dimensions valid. For every pruned channel k three tensors are sliced consistently: `conv1.weight[k]` (output filter), `bn1` weight/bias/running mean/running var at k, and `conv2.weight[:, k]` (the matching input slice). Hence conv1.out = bn1.channels = conv2.in in every block.

Reconstruction. A new, physically smaller `ResNet18(inner_channels=[...])` is instantiated and the surviving weights are copied in; no masks or zero channels exist at run time, and the channel configuration is stored in the checkpoint so it can be rebuilt anywhere. Correctness is asserted: the logits of the rebuilt network match the original network with the pruned channels zeroed to within 7e-7.

Block	Original hidden channels	Kept	Pruned
layer1.0	64	22	65.6%
layer1.1	64	27	57.8%
layer2.0	128	63	50.8%
layer2.1	128	57	55.5%
layer3.0	256	125	51.2%
layer3.1	256	122	52.3%
layer4.0	512	235	54.1%
layer4.1	512	309	39.6%

Directly after pruning (no training) accuracy falls to 23.39%, with parameters at 53.2% and MACs at 47.3% of the original. Fine-tuning for 3 epochs (LR 0.02, cosine) recovers it to **86.82%**. Checkpoint: `pruned_structured_resnet18.pth` (23.9 MB).

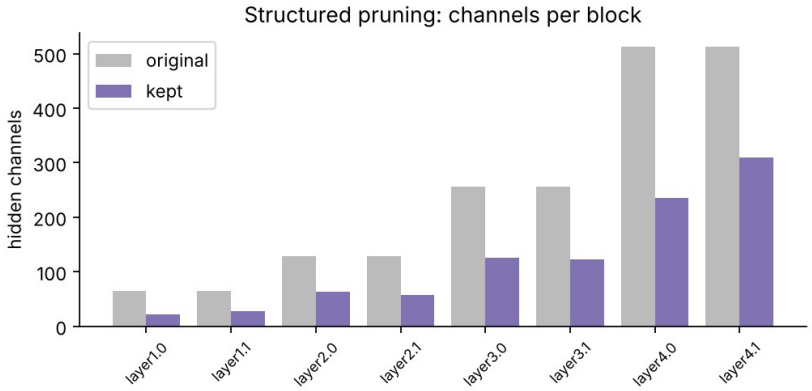


Figure 7 - Hidden channels per block before and after pruning

```
$ python src/prune_structured.py --ratio 0.5 --ft-epochs 3

=== Structured channel pruning | {'ratio': 0.5, 'min_keep': 0.25, 'ft_epochs': 3.0, 'lr': 0.02, 'fp32':
'/home/claude/sub/checkpoints/fp32_resnet18.pth', 'out':
'/home/claude/sub/checkpoints/pruned_structured_resnet18.pth', 'data_dir': '/home/claude/sub/data', 'seed': 42}
Baseline: acc 86.33% | params 11,173,962 | MACs 138.9 M
hidden channels per block (orig -> kept): ['64->22', '64->27', '128->63', '128->57', '256->125', '256->122',
'512->235', '512->309']
Reconstruction check: max |logit difference| vs. masked original = 7.15e-07
After pruning (no fine-tune): acc 23.39% | params 5,949,450 (53.2%) | MACs 65.6 M (47.3%)
  [structured] step  100/1170 loss 0.5848 | 0.7 min
  [structured] step  200/1170 loss 0.5277 | 1.4 min
  [structured] step  300/1170 loss 0.4888 | 2.1 min
  [structured] step  400/1170 loss 0.4863 | 2.7 min
  [structured] step  500/1170 loss 0.4403 | 3.4 min
  [structured] step  600/1170 loss 0.4269 | 4.1 min
  [structured] step  700/1170 loss 0.4091 | 4.7 min
  [structured] step  800/1170 loss 0.3848 | 5.3 min
  [structured] step  900/1170 loss 0.3502 | 6.0 min
  [structured] step 1000/1170 loss 0.3333 | 6.6 min
  [structured] step 1100/1170 loss 0.3433 | 7.3 min
  [structured] step 1170/1170 loss 0.3370 | 7.8 min
  [structured] done -> test acc 86.82%
Saved reconstructed model -> /home/claude/sub/checkpoints/pruned_structured_resnet18.pth (23.9 MB); final acc 86.82%
```

Screenshot 6 - Structured pruning console output

4.3 Fine-tuning pipeline (Q2.4)

`src/finetune.py` provides the shared routine (SGD + cosine decay, fractional epochs) used after every unstructured round and after structured pruning; it works both on masked models and on rebuilt smaller models and can also be run stand-alone on any float checkpoint.

4.4 Benchmark (Q2.6)

Model	Acc (%)	Non-zero params	Effective MACs (M)	File (MB)	gzip (MB)	Latency b=1 (ms)	Throughput (img/s)
Original FP32	86.33	11.17 M	138.9	44.78	41.30	6.39	818.8
Unstructured 85%	86.97	1.68 M	64.6 (sparse-aware)	44.78	9.95	7.36	894.5
Structured 50%	86.82	5.95 M	65.6	23.87	21.99	4.20	1453.6

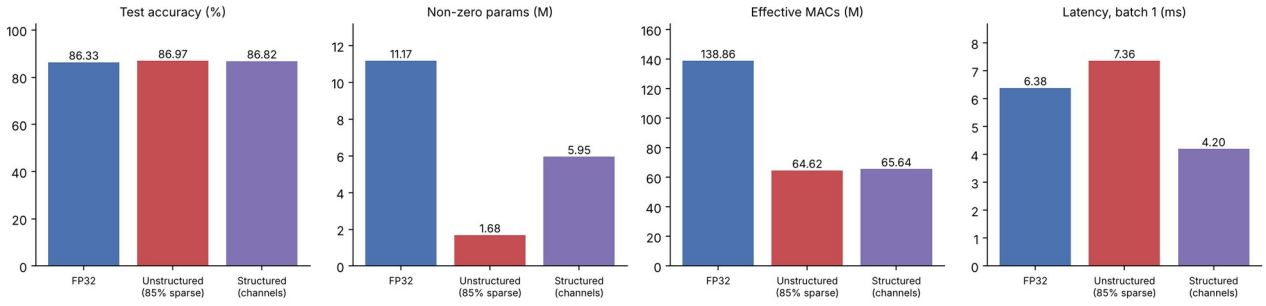


Figure 8 - Q2 comparison of the original, unstructured-pruned and structured-pruned models

Analysis. The two methods deliver very different kinds of benefit. Unstructured pruning reaches far higher compression (85% of weights are zero; 4.1x smaller after gzip and 5.5x smaller in the compact format) but the zeros are scattered, so dense CPU kernels do the same work: latency (7.4 ms) was no better than the original (6.4 ms) and slightly worse within measurement noise. Its saving is in storage and in the MACs a sparse-aware engine could skip, not in measured speed. Structured pruning removes whole channels, so the network is simply smaller and the speed-up is real on ordinary hardware: 1.5x lower batch-1 latency and 1.8x higher throughput, with 1.9x smaller files and 47% of the MACs. Its predictions agree with the original less often (92.6% vs 95.1% for unstructured), which is expected given the larger change to the architecture. Both recovered to at or slightly above baseline accuracy after fine-tuning; part of that gain is the additional training given to an under-trained baseline.

```
$ python src/benchmark_q2.py

       model  acc%    MB  gzMB  lat_ms  img/s  nz_params  MACs_M  sparse%  lat_x  thr_x
        fp32 86.33 44.78 41.30   6.385  818.8   11173962  138.86     0.00   1.00   1.00
unstructured 86.97 44.78  9.95   7.358  894.5    1684263   64.62    84.93   0.87   1.09
  structured 86.82 23.87 21.99   4.198 1453.6    5949450   65.64     0.00   1.52   1.78
```

Screenshot 7 - Q2 benchmark output

5. ONNX deployment (Deliverable 8)

Script: `src/export_onnx.py` (and the QAT export inside `qat.py`); CPU inference and benchmarking: `src/onnx_inference.py` with ONNX Runtime (CPUExecutionProvider).

- **FP32, unstructured, structured:** direct `torch.onnx.export` (opset 13, dynamic batch). ONNX Runtime outputs match PyTorch to about 2e-6.
- **PTQ-INT8:** the FP32 ONNX graph is statically quantized with ONNX Runtime (QDQ format, per-channel INT8 weights, the same 1,000-image calibration subset). PyTorch eager INT8 modules use fbgemm packed ops that have no ONNX equivalent, so this is the standard route to an INT8 ONNX file.
- **QAT:** the trained fake-quant model is exported with BatchNorm folded into the conv weights and every fake-quant turned into QuantizeLinear/DequantizeLinear (51 pairs). The 0-127 "reduce_range" activation grid used by fbgemm is re-expressed on the 0-255 range (same real-valued clip range) because ONNX cannot express 7-bit grids. Its ONNX accuracy is 87.18%, equal to the fake-quant model.

ONNX model	Acc (%)	Size (MB)	Latency b=1 (ms)	Throughput (img/s)
FP32	86.33	44.69	3.01	658.0
PTQ-INT8	86.35	11.29	0.87	2224.9
QAT (QDQ)	87.18	44.74	0.86	2276.5
Unstructured	86.97	44.69	3.28	701.1
Structured	86.82	23.79	2.01	854.6

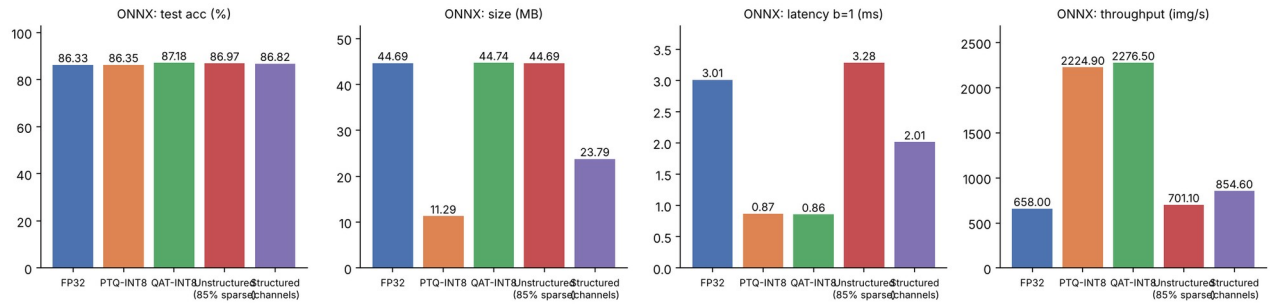


Figure 9 - ONNX Runtime CPU results

Notes. ONNX Runtime batch-1 FP32 latency (3.0 ms) is lower than PyTorch eager (6.4 ms), so ONNX speed-ups of INT8 are relative to a faster float baseline (3.5x). The QAT ONNX file keeps float weights inside QDQ nodes and so is 44.7 MB; the 11.4 MB INT8 form of the QAT model is the TorchScript file. The ONNX numbers come from a different runtime than Q1/Q2 and should not be mixed with them.

```
$ python src/onnx_inference.py

       model  acc%    MB  gzMB  lat_ms  img/s  lat_x  thr_x
        fp32 86.33 44.69 41.31   3.010  658.0   1.00   1.00
         ptq 86.35 11.29 10.36   0.870 2224.9   3.46   3.38
         qat 87.18 44.74 41.31   0.856 2276.5   3.52   3.46
unstructured 86.97 44.69  9.93   3.281  701.1   0.92   1.07
  structured 86.82 23.79 21.99   2.013  854.6   1.50   1.30
```

Screenshot 8 - ONNX benchmark output

```
$ python src/inference.py --model all --num-images 5


=== model: fp32 | backend: torch | file: fp32_resnet18.pth
  # prediction     conf  truth       ok
  0 cat           88.0%  cat         Y
  1 ship          96.8%  ship        Y
  2 ship          95.6%  ship        Y
  3 airplane      93.9%  airplane    Y
  4 frog          99.9%  frog        Y
accuracy on these 5 images: 100.0%

=== model: ptq | backend: torch | file: ptq_int8_resnet18.pt
  # prediction     conf  truth       ok
  0 cat           86.8%  cat         Y
  1 ship          96.7%  ship        Y
  2 ship          94.6%  ship        Y
  3 airplane      93.8%  airplane    Y
  4 frog          99.9%  frog        Y
```

Screenshot 9 - Inference pipeline demo (all five models)

6. Summary and limitations

Model	Acc (%)	Size	Main benefit
FP32 baseline	86.33	44.8 MB	reference
PTQ-INT8	86.30	11.4 MB	3.9x smaller, 5.5x faster b=1, no retraining
QAT-INT8	87.30	11.4 MB	3.9x smaller, 4.9x faster b=1, trained for INT8
Unstructured 85%	86.97	8.2 MB compact	4-5x smaller storage, no speed-up on dense kernels
Structured 50%	86.82	23.9 MB	1.9x smaller, 1.5x faster b=1, 53% of MACs

- The baseline is intentionally short-trained (8 epochs, stride-2 stem) because of CPU-only training; accuracies are therefore below the ~94-95% of a fully trained CIFAR ResNet-18. The relative comparisons are what this report supports.
- Accuracy gains of QAT and the pruned models over FP32 are largely due to their extra fine-tuning epochs and should not be attributed to compression.
- Each configuration was run once with seed 42, so differences of a few tenths of a point (such as between the PTQ calibration sizes) are within run-to-run noise.
- Structured pruning removes only block-internal channels; residual-stream channels are kept. Pruning those would need dependency-group handling across blocks and is a natural extension.
- Eager-mode quantization in torch.ao is deprecated in newer PyTorch in favour of torchao, but remains functional in the tested version.

7. Reproduction

See `README.md`. In short: `pip install -r requirements.txt` then `./run_all.sh` (about 2 hours on 2 CPU cores; `QUICK=1 ./run_all.sh` runs a few-minute smoke test).